



UNIVERSIDAD NACIONAL DE AVELLANEDA

DPTO. DE TECNOLOGÍA Y ADMINISTRACIÓN
INGENIERÍA EN INFORMÁTICA

Sistemas de Control Automático

Trabajo de Laboratorio 1

Control Encendido-Apagado de un Ascensor

ESTUDIANTES

Massimino, Matías

Colli, Alejandro

Cuocina, Matías

Eroles, Pedro

DOCENTE

Guillermo Federico Caporaletti

2do Cuatrimestre 2023

APROBADO 6/6

Contenido 3/3
Presentación 1/1
Redacción 1/1
Plazo 1/1

GF Caporaletti, 23/10/2023

Índice

Objetivo	2
Introducción	3
Modelo del Ascensor.....	4
Equipamiento y componentes utilizados	6
Arduino Uno.....	6
Modulo Puente H L298n	6
Optoacoplador	7
Pulsador	10
Desarrollo	12
Medición.....	15
Conclusión.....	25
Mejoras a desarrollar.....	26
Anexo	27
Bibliografía	31

Objetivo

El objetivo de este trabajo de laboratorio era controlar un sistema físico, en nuestro caso, elegimos un ascensor, mediante una acción de control de encendido-apagado (ON-OFF).

La experiencia se registró meticulosamente para medir la respuesta del sistema y evaluar la reacción del sistema ante perturbaciones. Buscábamos encontrar la máxima precisión posible con este modo de acción de control.

Introducción

Para el presente trabajo de laboratorio decidimos en grupo optar por realizar el control on-off de un ascensor.

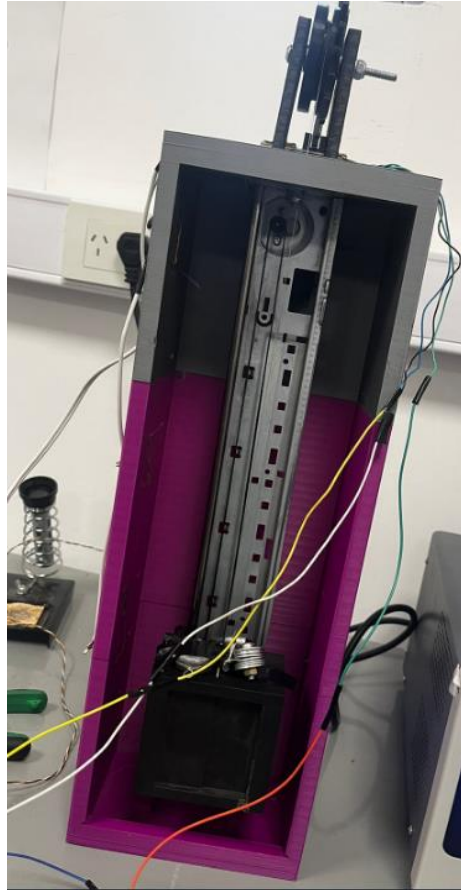


Figura 1: Fotografía del ascensor

Nuestro objetivo será controlar dicho sistema físico de tipo mecánico con una acción de control respecto a la altura a través de una placa Arduino.

Se buscará la máxima precisión posible con este modo de acción de control. Es por ello que, planteamos realizar variaciones en los parámetros a manipular para reconfigurar la experiencia y realizar diferentes mediciones con la finalidad de analizar y comparar los resultados obtenidos.

Modelo del Ascensor

Para plantear la transferencia del sistema, lo que tuvimos que hacer fue tomar como variable controlada a la velocidad ya que, si utilizáramos la altura, nos quedaba reflejado un sistema de segundo orden.

Para esto planteamos lo siguiente:

$$F_r = -b * V(t)$$

F_r representa la fuerza de rozamiento, b es la constante de rozamiento mientras que $v(t)$ es la velocidad del ascensor en función del tiempo.

$$F = F_m + F_r = m * \frac{dv}{dt}$$

F representa la fuerza total que actúa que en este caso es la fuerza de rozamiento sumado a la fuerza motriz. m representa la masa del ascensor.

$$K * E(t) - b * V(t) = m * \frac{dv}{dt}$$

$$K * E(t) = m * \frac{dv}{dt} + b * v(t)$$

Transformamos la ecuación a un dominio de Laplace, donde tenemos la variable compleja s .

$E(s)$ y $V(s)$ son las transformadas de Laplace de las funciones $E(t)$ y $V(t)$ respectivamente

$$K * E(s) = (ms + b) * V(s)$$

Finalmente llegamos a la transferencia de nuestro sistema que relaciona la velocidad y la tensión.

$$G(s) = \frac{V(s)}{E(s)} = \frac{K_v}{\tau * s + 1}$$

Tenemos $V(s)$ que representa la velocidad, y $E(s)$ la tensión aplicada.

K_v representa la constante de velocidad / voltio. Es decir, la velocidad a la que llegará el ascensor por cada voltio que se le aplique al motor.

Tenemos la masa del ascensor que afecta la inercia, es decir, su resistencia al cambio en la velocidad. La masa va a disminuir su aceleración.

La velocidad máxima depende de la constante de rozamiento y la fuerza contra electromotriz. Y esto se da cuando $F = F_r$.

Al utilizar la altura para plantear la transferencia nos había quedado lo siguiente, y por eso optamos por la velocidad para simplificar problemas.

$$H(s) = \frac{A(s)}{E(s)} = \frac{1}{s} * G(s)$$

$A(s)$ es la altura, que es igual a la integral de la velocidad. Así queda definido nuestro sistema de segundo orden.

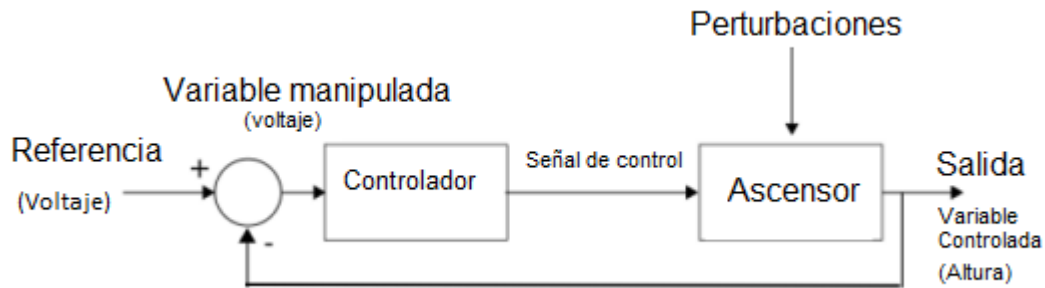


Figura 2: Diagrama en bloques de nuestro sistema

El sistema que estamos controlando se puede representar en un diagrama en bloques, como se observa en la figura anterior. En nuestro sistema, la planta va a ser el ascensor, el cual va a contener un motor y un optoacoplador; la variable controlada va a ser la altura del ascensor; y finalmente la variable manipulada va a ser el voltaje que reciba el motor para subir el ascensor.

Como nuestro sistema tiene una acción de control ON-OFF, podemos decir que no es lineal, como todos los sistemas de control ON-OFF. Esto es así ya que el sistema no cumple con el principio de superposición, que establece que la suma de dos entradas tiene que ser igual a la suma de las salidas de cada entrada por separado.

También podemos afirmar que nuestro sistema es invariante en el tiempo, ya que, aunque posee componentes que son afectados por su uso, corrosión o envejecimiento del material, no consideramos que son suficientemente importantes para exigir ser considerados en nuestro diseño.

Equipamiento y componentes utilizados

Arduino Uno

El Arduino Uno es una placa de microcontrolador de código abierto basado en el microchip ATmega328P y desarrollado por Arduino.cc. La placa tiene 14 pines digitales, 6 pines analógicos y programables con el Arduino IDE (Entorno de desarrollo integrado) a través de un cable USB tipo B. Puede ser alimentado por el cable USB o por una batería externa de 9 voltios, aunque acepta voltajes entre 7 y 20 voltios.



Figura 3: Placa Arduino Uno utilizada para desarrollar el trabajo

Arduino IDE es de código abierto, su código fuente está alojado en GitHub.

Modulo Puente H L298n

Este módulo posee dos puentes H que permiten controlar 2 motores DC o un motor paso a paso bipolar/unipolar.

El módulo permite controlar el sentido de giro y velocidad mediante señales TTL que se pueden obtener de microcontroladores y tarjetas de desarrollo como Arduino, Raspberry Pi o Launchpads de Texas Instruments.

Tiene integrado un regulador de voltaje de 5V encargado de alimentar la parte lógica del L298N, el uso de este regulador se hace a través de un Jumper y se puede usar para alimentar la etapa de control.

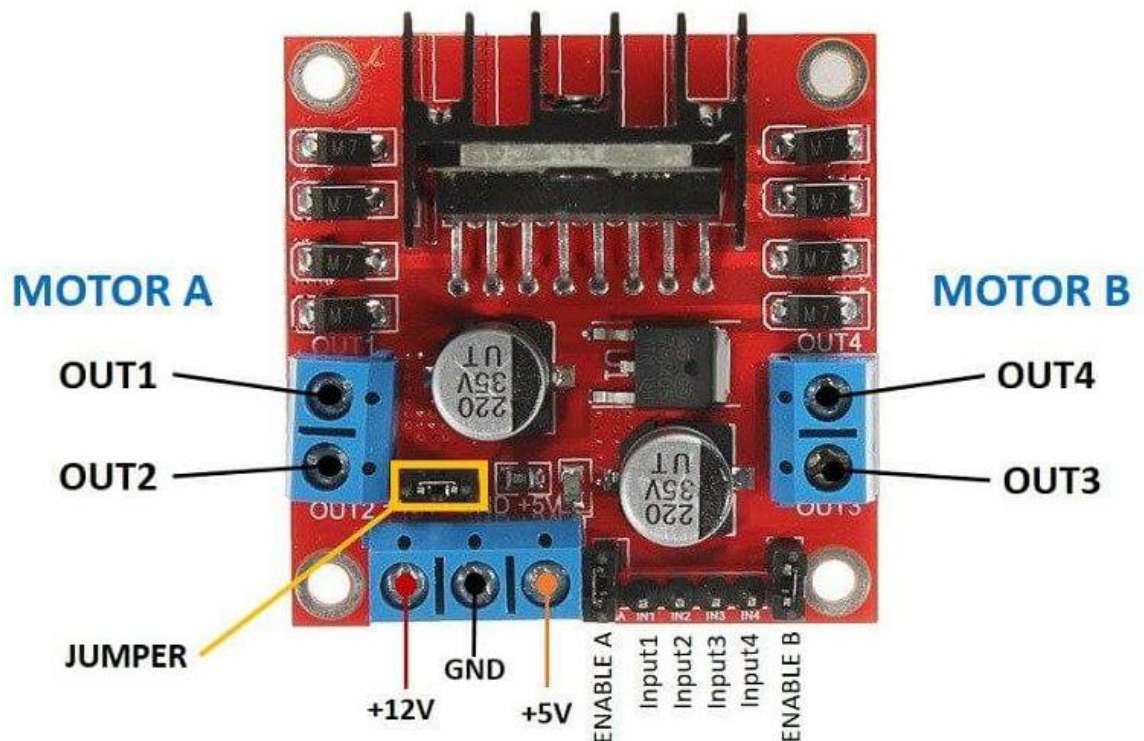


Figura 4: Puente H L298n utilizado.

El módulo L298N se alimenta de una fuente de tensión externa. Cada medio puente requiere 3 pines de salida de un microcontrolador.

Optoacoplador

Este módulo usa un sensor de acoplamiento óptico de ranura importado, con un ancho de 5 mm. El principio de funcionamiento del acoplador óptico es que el foto acoplador produce una corriente óptica debido al efecto fotoeléctrico, que se induce a partir de la salida del fotón y realiza la conversión de luz-electricidad. El acoplador óptico (acoplador, OC) también

se conoce como sensor fotoeléctrico o foto acoplamiento para abreviar. El foto acoplador transmite señales eléctricas utilizando luz como medio. Los acopladores ópticos se componen generalmente de tres partes: la emisión de luz, la recepción de luz y la amplificación de la señal.

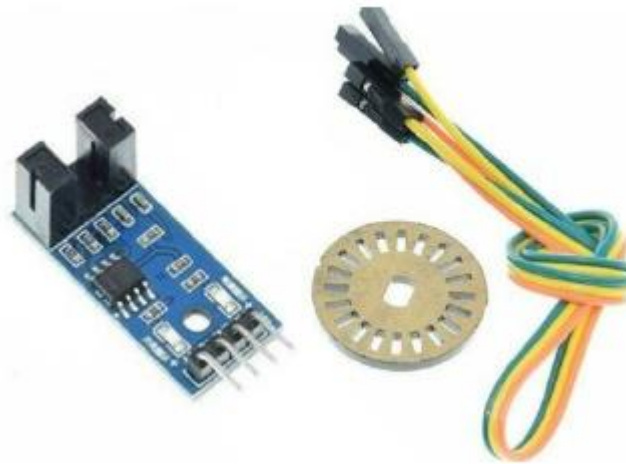


Figura 5: Optoacoplador

En nuestro caso, utilizamos un optoacoplador de diseño específico de HP sobre el cual no tenemos una documentación, pero tenemos la siguiente descripción.

Cuando la polea del ascensor pasa a través de la ranura del optoacoplador, el LED dentro del optoacoplador emite luz que incide directamente sobre el fototransistor. Esto activa el fototransistor y genera una señal eléctrica. La duración y el momento en que esta señal se activa proporcionan información sobre la posición actual del ascensor.

Los estados del optoacoplador se registraron y se utilizaron para determinar la posición del ascensor en tiempo real.

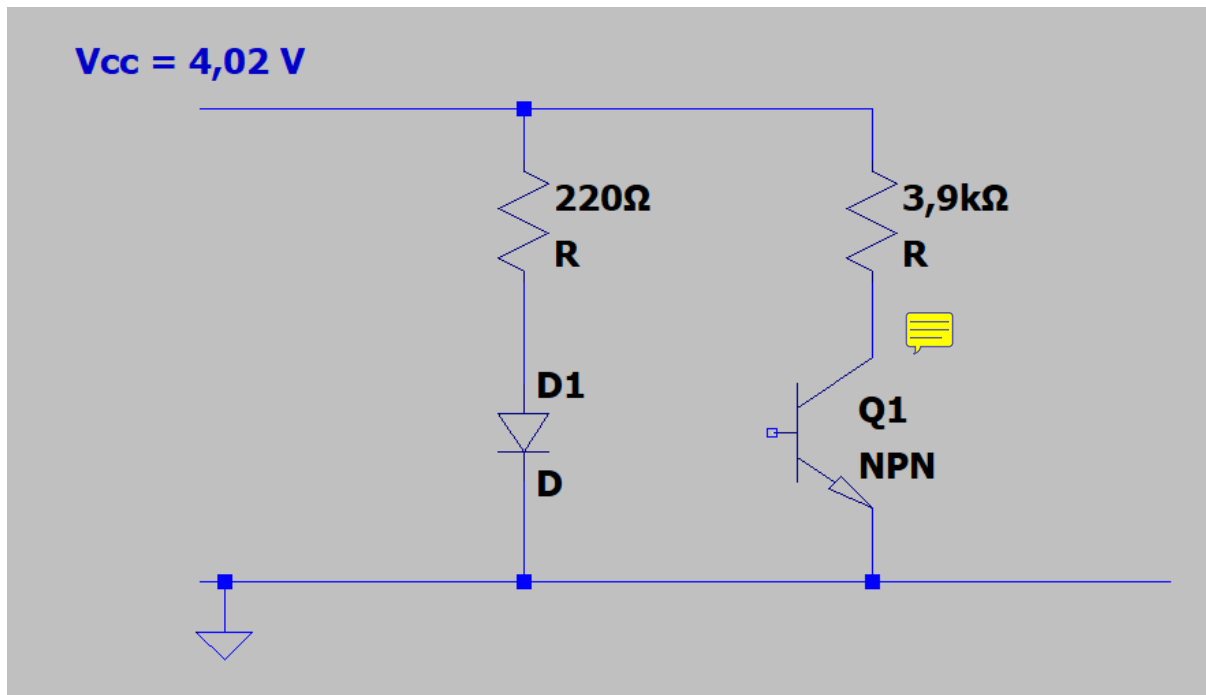


Figura 6: Esquemático de circuito optoacoplador utilizado.

Como vemos, tenemos nuestras dos resistencias, el diodo y el fototransistor. El optoacoplador fue conectado a los 5V de la fuente de Arduino. El cable verde fue conectado al negativo del protoboard. El fototransistor es una fuente de corriente, **pero no continua. Entrega** corriente cuando recibe luz.

También se podría usar como un interruptor óptico. El funcionamiento básico de este sensor es el siguiente; Si se hace pasar cualquier cosa entre la ranura del sensor, este crea un pulso digital en el pin D0. Este pin, es el colector del fototransistor de la figura 6. Es el punto entre el fototransistor y la resistencia de $3,9k$.

Este pulso va de 0V a 5V y es una señal digital TTL. Luego con Arduino podemos leer este pulso. Conocer la posición o velocidad de un motor es muy importante en robótica, para lo cual existen diversas alternativas, siendo una de las más comunes el uso de encoders de tipo óptico. Los encoders en general son dispositivos que se encargan de convertir el movimiento angular o lineal en pulsos eléctricos que puedan ser interpretados por el controlador del sistema. Los encoders incrementales ópticos realizan la medición de movimiento con el uso de un haz de luz infrarrojo que se ve interrumpido por

las ranuras de un disco acoplado al eje. La cantidad de ranuras por vuelta determinará la precisión del encoder.

Pulsador

Un pulsador electrónico, también conocido como interruptor electrónico o botón electrónico, es un dispositivo utilizado para controlar circuitos electrónicos.

Para realizar nuestro TL, optamos por un pulsador electromecánico tradicional, que utiliza contactos físicos para abrir o cerrar un circuito eléctrico.



Figura 7: Botón pulsador

Un pulsador electrónico generalmente consta de dos terminales o contactos, que se conectan a los extremos de un circuito. Cuando se presiona el botón o se activa el interruptor, se produce un cambio en la configuración interna del dispositivo, permitiendo que la corriente eléctrica fluya a través de él o se interrumpa.

El modo de configuración de los pulsadores se determina con la resistencia conocida como pull-down o pull-up. Como funcionalidad básica, estas resistencias establecen un estado lógico en un pin o entrada de un circuito lógico cuando se encuentra en estado de reposo. Como bien indica su nombre la resistencia pull up establece un estado HIGH y las resistencias pull down establecen un estado LOW cuando el pin se encuentra en reposo. Esto evita los falsos estados que se producen por el ruido generado por los circuitos electrónicos.

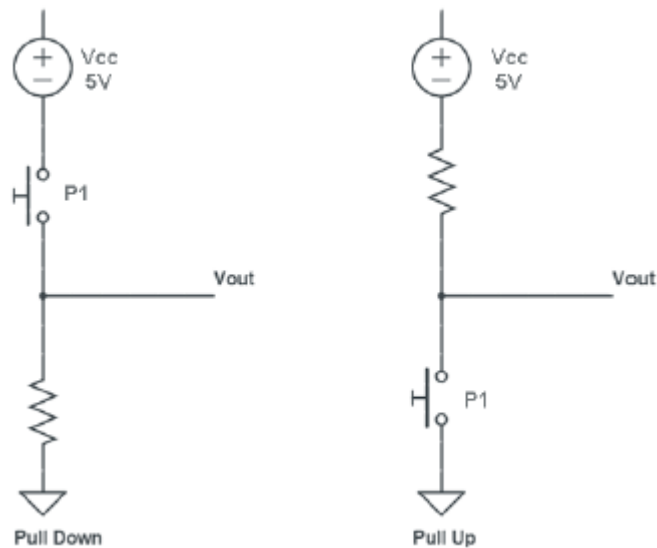


Figura 8: Configuración de resistencia en Pull Down o Pull Up.

En el caso nuestro, para realizar el trabajo de laboratorio optamos por una configuración Pull Up.

Estas dos configuraciones nos evitarán que en estado de reposo midamos un valor erróneo eliminando la influencia de factores externos sobre nuestras mediciones como el ruido eléctrico.

Fuente de Tensión



Figura 9: Fuente de alimentación.

Desarrollo

Para poder llevar adelante el trabajo de laboratorio, lo primero que tuvimos que hacer fue reformar nuestro sistema, ya que al tratarse de un ascensor nos dimos cuenta de que la polea debía ser reformada para el correcto uso y funcionamiento del optoacoplador.

Para eso creamos un sistema en donde tendremos:

Polea: Colocada en el ascensor de manera que pueda pasar a través del optoacoplador.

Optoacoplador: Montado en una posición estratégica donde la polea pasa a través de su ranura o apertura.

Soporte: Diseñada para sostener la polea, fijarla al ascensor y poder pasar el optoacoplador.

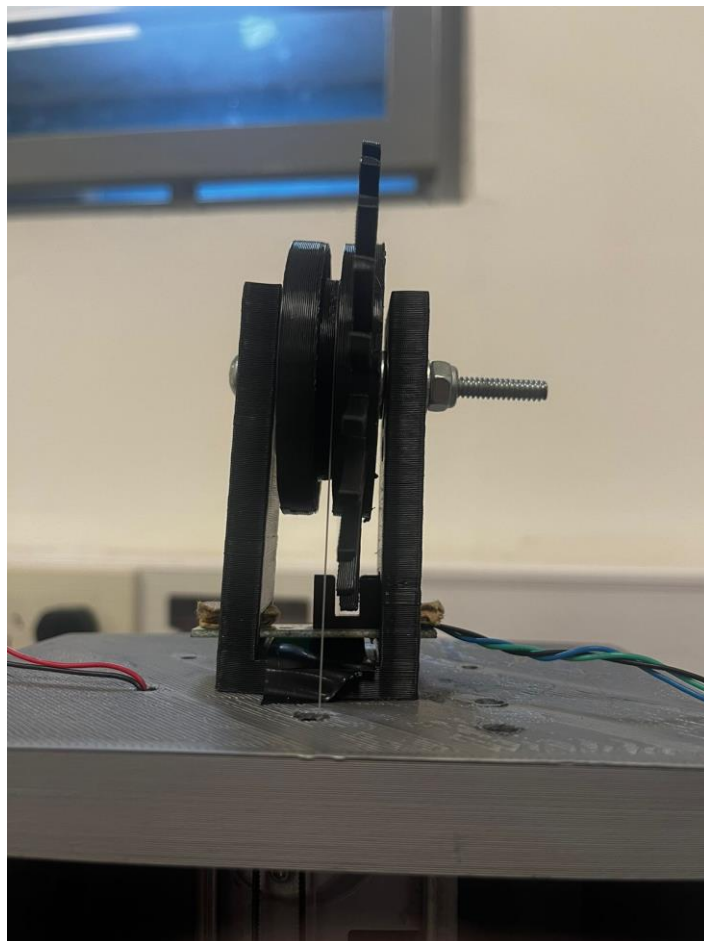


Figura 10: Sistema de polea del ascensor para medir los estados del optoacoplador.

Luego, nos pusimos a realizar el armado del código para que pueda funcionar nuestro sistema. Para desarrollar el código utilizamos la plataforma Arduino.

Además, se estableció una conexión en la entrada del circuito con un puente H para permitirle al motor girar en ambos sentidos, avance y retroceso.

Por último, se conectó también a la entrada del arduino para realimentar.

El puente H se conectó al positivo de la fuente, el GND se conectó a la entrada negativa del protoboard, a su vez, la salida del motor A del puente H se conectó al motor. Los pines 1 y 2, a las conexiones 12 y 13 del arduino.

El arduino se conectó al USB de la notebook, las conexiones 12 a 13 mencionados anteriormente al puente H, el conector 5v fue conectado al positivo del protoboard, el GND del arduino al negativo del protoboard.

Una vez finalizadas las conexiones, nos quedó de la siguiente manera.

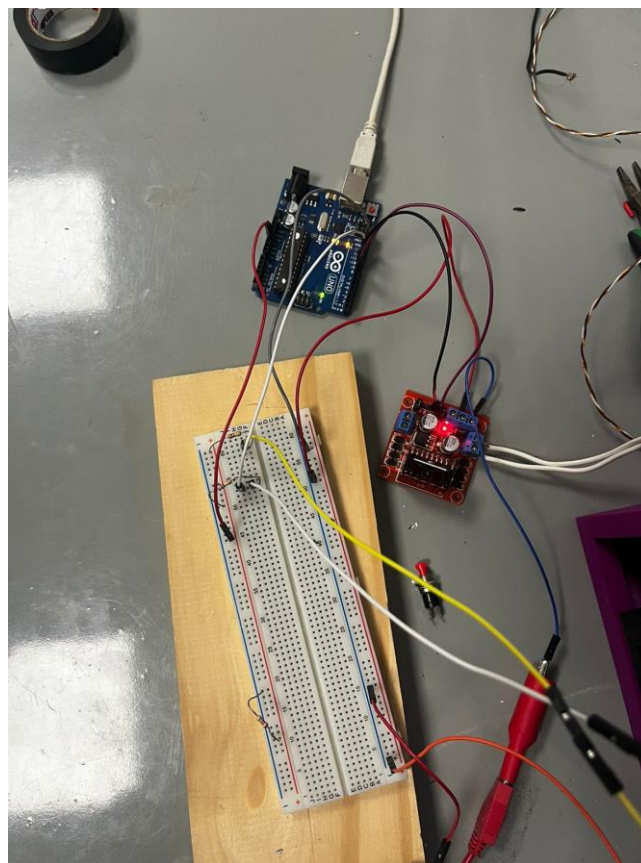


Figura 11: Conexión final

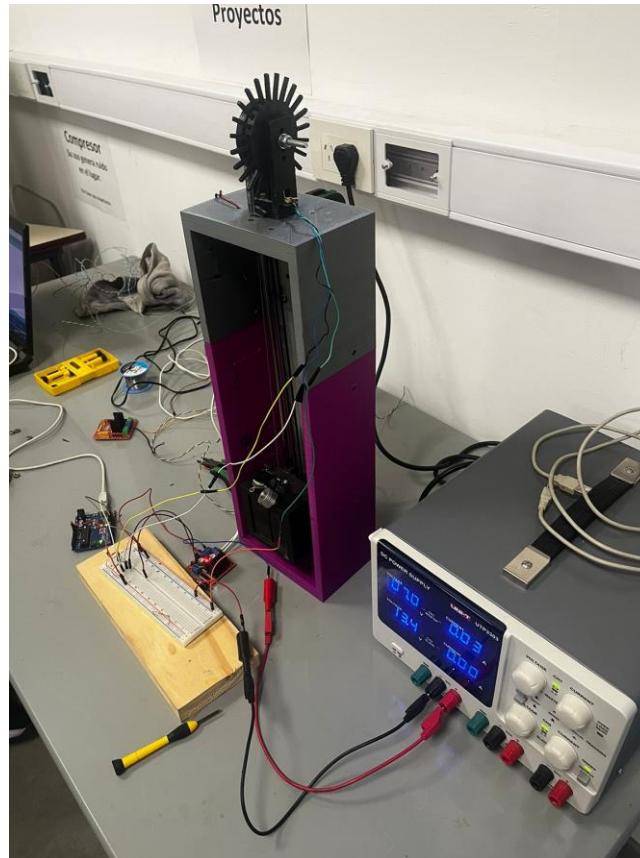


Figura 12: Conexión final.

Para el desarrollo de nuestro proyecto, contamos con un control ON-OFF, que a partir de las funciones “controlar” y “actuar” definidas en el código, decidimos si el motor debe subir, bajar o detenerse en función de la posición en la que se encuentre y la posición establecida como objetivo.

Medición

Ya habiendo hecho el código y las conexiones de nuestro sistema, podemos pasar a la experiencia en sí. Lo que hicimos fue realizar una medición, poniendo una altura objetivo para la subida, tanto como para la bajada. A partir de los datos obtenidos de la misma medición, pudimos calcular los parámetros del sistema.

Nuestra medición la realizamos cada 100 ms, establecimos un objetivo de 270 mm para la subida, y un objetivo de 60 mm para la bajada.

También, establecimos un margen de error de $\pm 10mm$. Podemos ver que inicialmente, el ascensor se encuentra en una altura de $0mm$ y un estado = 0, que significa que el motor está apagado. Configuramos en el código para que este estado cambie a los $3000ms$ y la altura rápidamente cambia y empieza a subir el ascensor. Finalmente, llegamos a un valor de $275,52mm$, nuevamente cambia el estado a 0, y luego a los $6199ms$ cambia el estado y comienza a bajar. Finalmente baja hasta los $63,14mm$.

Con los datos obtenidos de esta medición, ahora podemos pasar a obtener los parámetros de nuestro sistema tanto para la subida como para la bajada.

“Tabla 1: Datos recopilados durante la subida del ascensor”;

Tiempo [ms]	Altura [mm]	Altura Obj [mm]
2999	-	-
3099	-	-
3199	5,74	270,00
3299	11,48	270,00
3399	22,96	270,00
3499	40,18	270,00
3599	57,40	270,00
3699	74,62	270,00
3799	103,32	270,00
3899	132,02	270,00
3999	160,72	270,00
4099	186,55	270,00
4199	212,38	270,00
4299	241,08	270,00
4399	269,78	270,00
4499	275,52	270,00
4599	275,52	270,00
4699	275,52	270,00
4799	275,52	270,00
4899	275,52	270,00
4999	275,52	270,00
5099	275,52	270,00
5199	275,52	270,00
5299	275,52	270,00
5399	275,52	270,00
5499	275,52	270,00
5599	275,52	270,00
5699	275,52	270,00
5799	275,52	270,00
5899	275,52	270,00
5999	275,52	270,00
6099	275,52	270,00

“Tabla 2: Datos recopilados durante la bajada del ascensor”;

Tiempo [ms]	Altura [mm]	Altura Obj [mm]
6199	269,78	60,00
6299	252,56	60,00
6399	232,47	60,00
6499	209,51	60,00
6599	183,68	60,00
6699	149,24	60,00
6799	109,06	60,00
6899	68,88	60,00
6999	57,40	60,00
7099	71,75	60,00
7199	63,14	60,00
7299	63,14	60,00
7399	63,14	60,00
7499	63,14	60,00
7599	63,14	60,00
7699	63,14	60,00
7799	63,14	60,00
7899	63,14	60,00
7999	63,14	60,00
8099	63,14	60,00
8199	63,14	60,00
8299	63,14	60,00
8399	63,14	60,00
8499	63,14	60,00
8599	63,14	60,00
8699	63,14	60,00
8799	63,14	60,00
8899	63,14	60,00
8999	63,14	60,00
9099	63,14	60,00
9199	63,14	60,00
9299	63,14	60,00
9399	63,14	60,00
9499	63,14	60,00
9599	63,14	60,00
9699	63,14	60,00
9799	63,14	60,00
9899	63,14	60,00

Vemos en estas imágenes que el ascensor permanece detenido en posición 0 hasta los 3099 *ms* y luego comienza a subir.

Para obtener los parámetros de subida planteamos el siguiente gráfico en Excel a partir de los datos obtenidos.

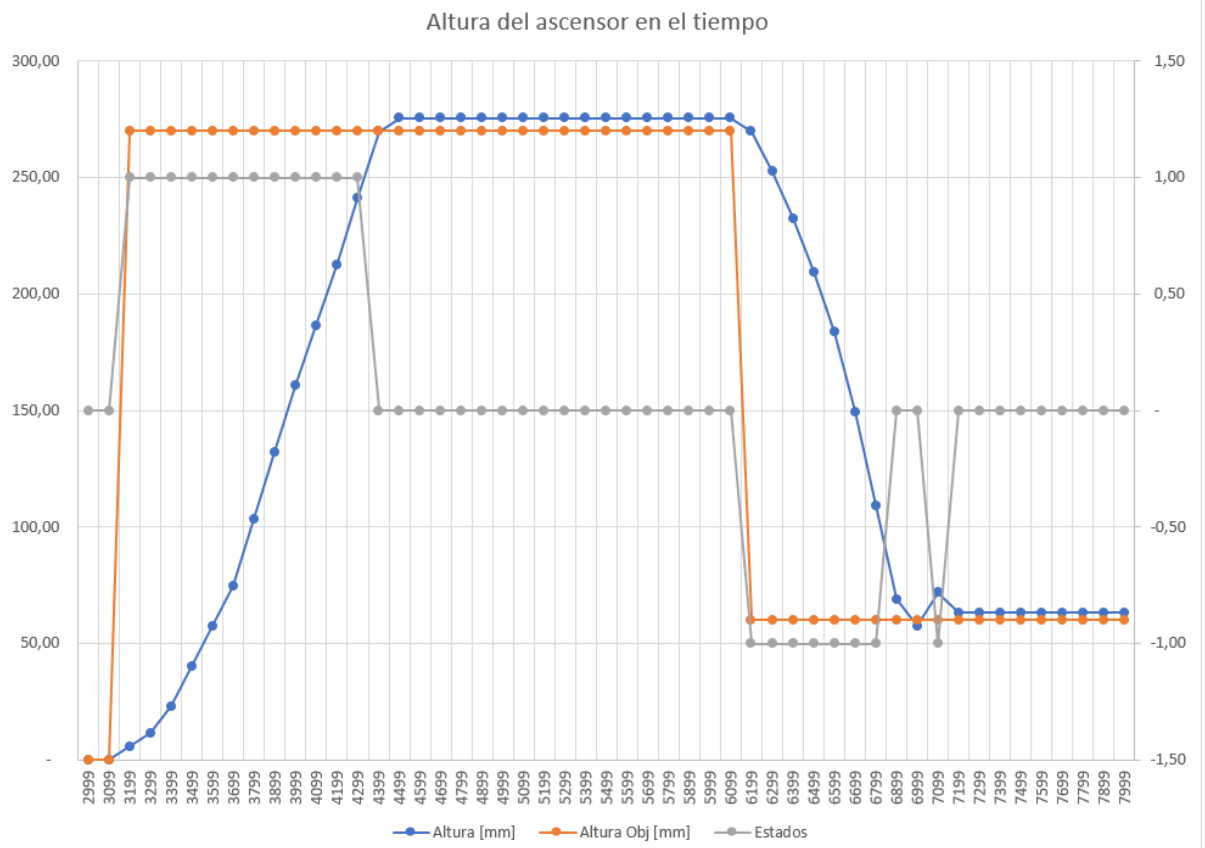


Figura 13: Altura del ascensor en el tiempo.

Planteamos lo siguiente

$$G(Ascensor) = \frac{V(s)}{E(s)} = \frac{K_v}{1 + \tau * s}$$

Tenemos a $V(s)$ que representa la velocidad, mientras que $E(s)$ representa la tensión.

Ponemos $6V$, ya que nos dimos cuenta que $1V$ caía sobre el puente H.

Para obtener la velocidad máxima a partir del gráfico podemos trazar la siguiente recta.

Trabajo de Laboratorio 1: Control Encendido-Apagado de un Ascensor

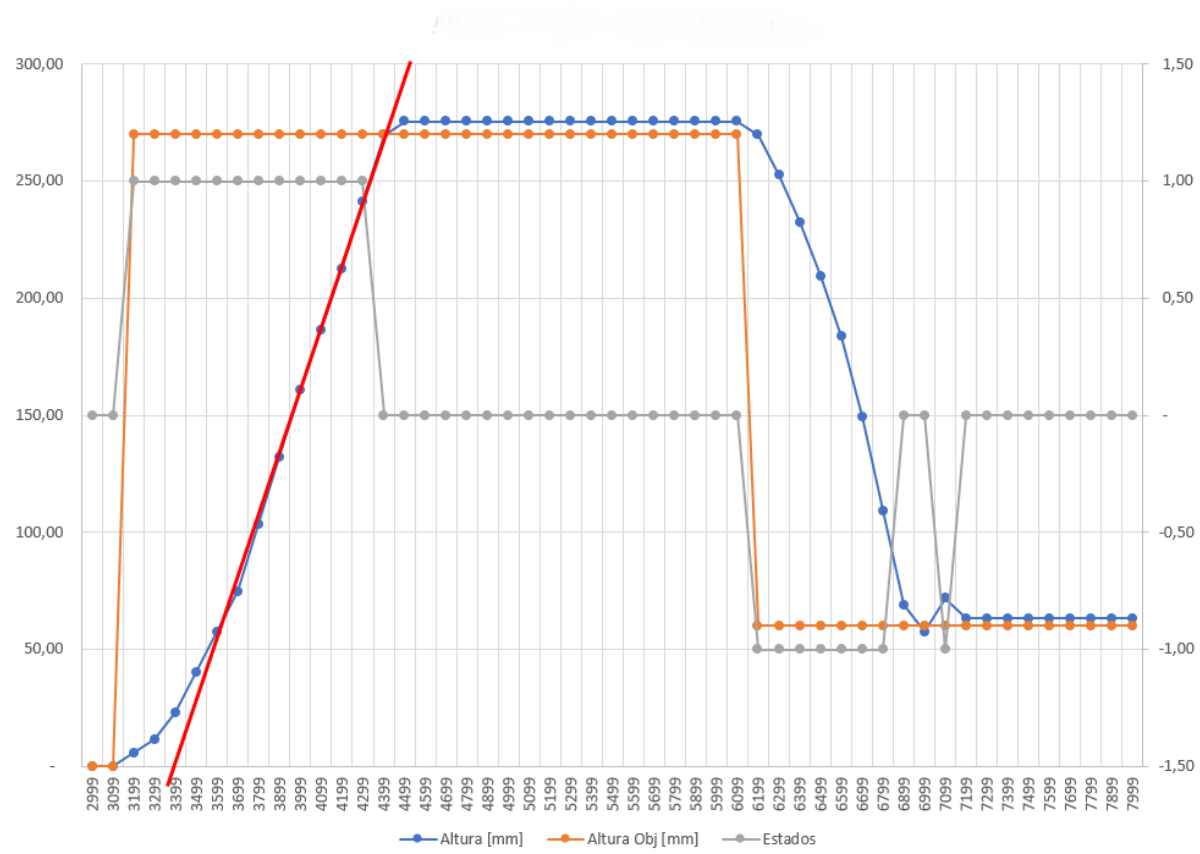
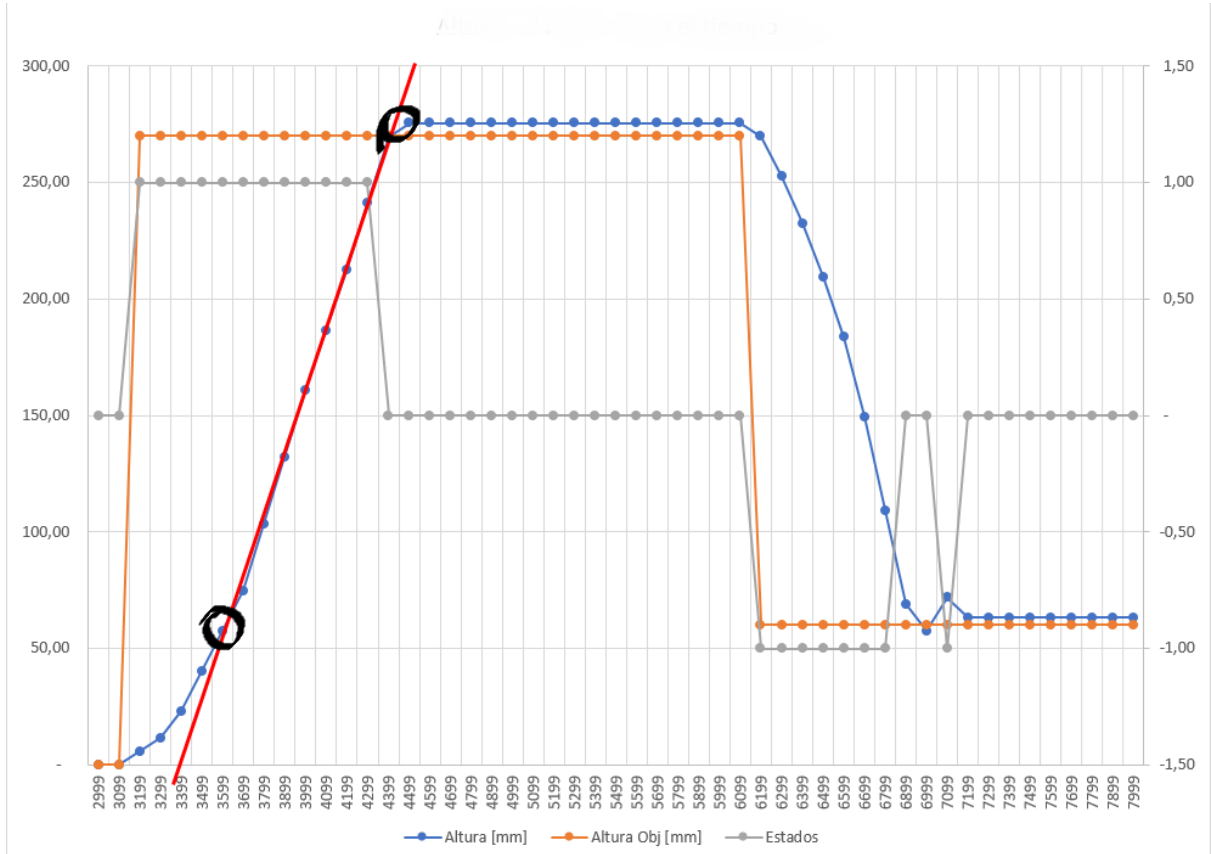


Figura 14: Velocidad máxima del ascensor.



Tomamos todos los datos que hay entre estos dos puntos y los

metemos en otra tabla de Excel para analizar el valor de ese K'.

“Tabla 3: Datos utilizados para calcular la velocidad de subida”;

Tiempo [s]	Altura [mm]
3,699	74,62
3,799	103,32
3,899	132,02
3,999	160,72
4,099	186,55
4,199	212,38
4,299	241,08
4,399	269,78

Utilizando estos datos obtenemos el siguiente gráfico.

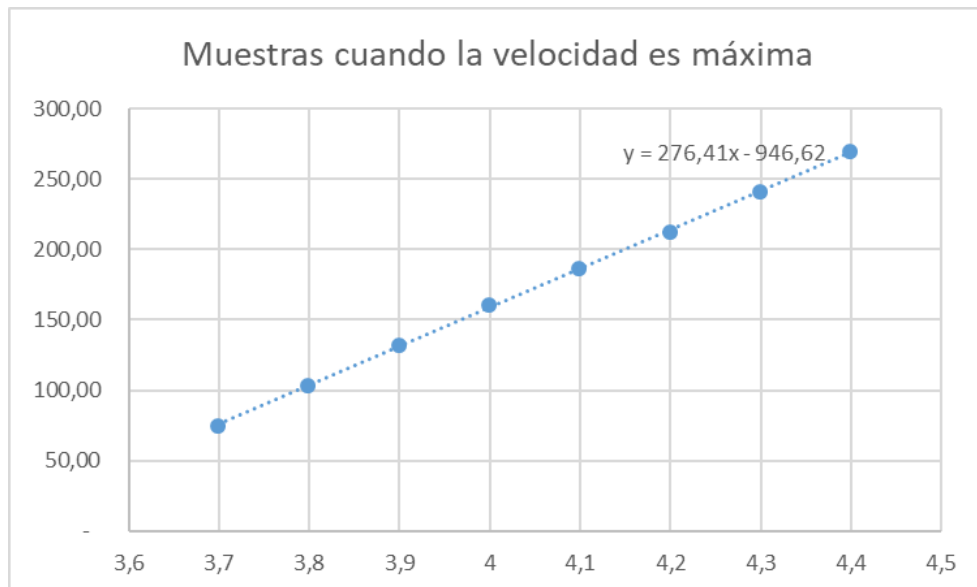


Figura 15: Muestras cuando la velocidad es máxima.

Como vemos, a partir de la recta, tenemos que la velocidad máxima es de 27,6 cm/s.

Ahora para obtener el tau podemos plantear lo siguiente.

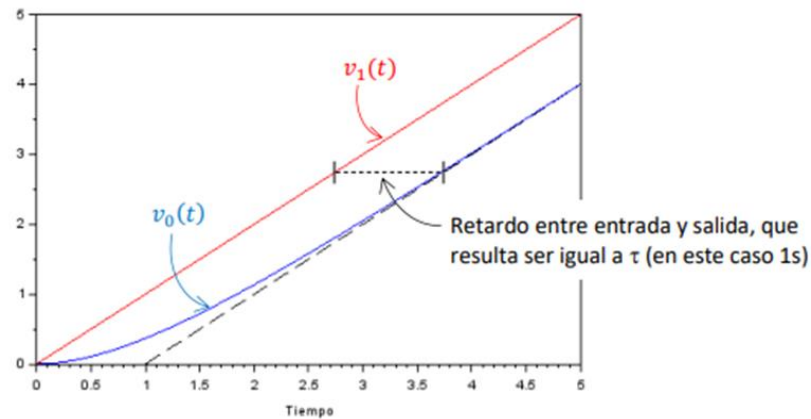


Figura 16: Ejemplo para el cálculo del tiempo natural de crecimiento.

Vemos que se parece bastante al gráfico que tenemos nosotros, ya que vemos que la rampa está en el hecho de que la posición es la integral de la velocidad.

Entonces tenemos que cuando $y = 0$, el tiempo es de 3,4246 s.

El tau va a ser la diferencia entre este tiempo y el tiempo que tardó en empezar a subir.

$$\tau = 3,425 \text{ s} - 3,099 \text{ s}$$

$$\tau = 326 \text{ ms}$$

Ya tenemos nuestros parámetros durante la subida.

$$V_{max} = 27,6 \text{ cm/s}$$

$$\tau = 326 \text{ ms}$$

Ahora debemos calcular los parámetros durante la bajada del ascensor.

Trabajo de Laboratorio 1: Control Encendido-Apagado de un Ascensor

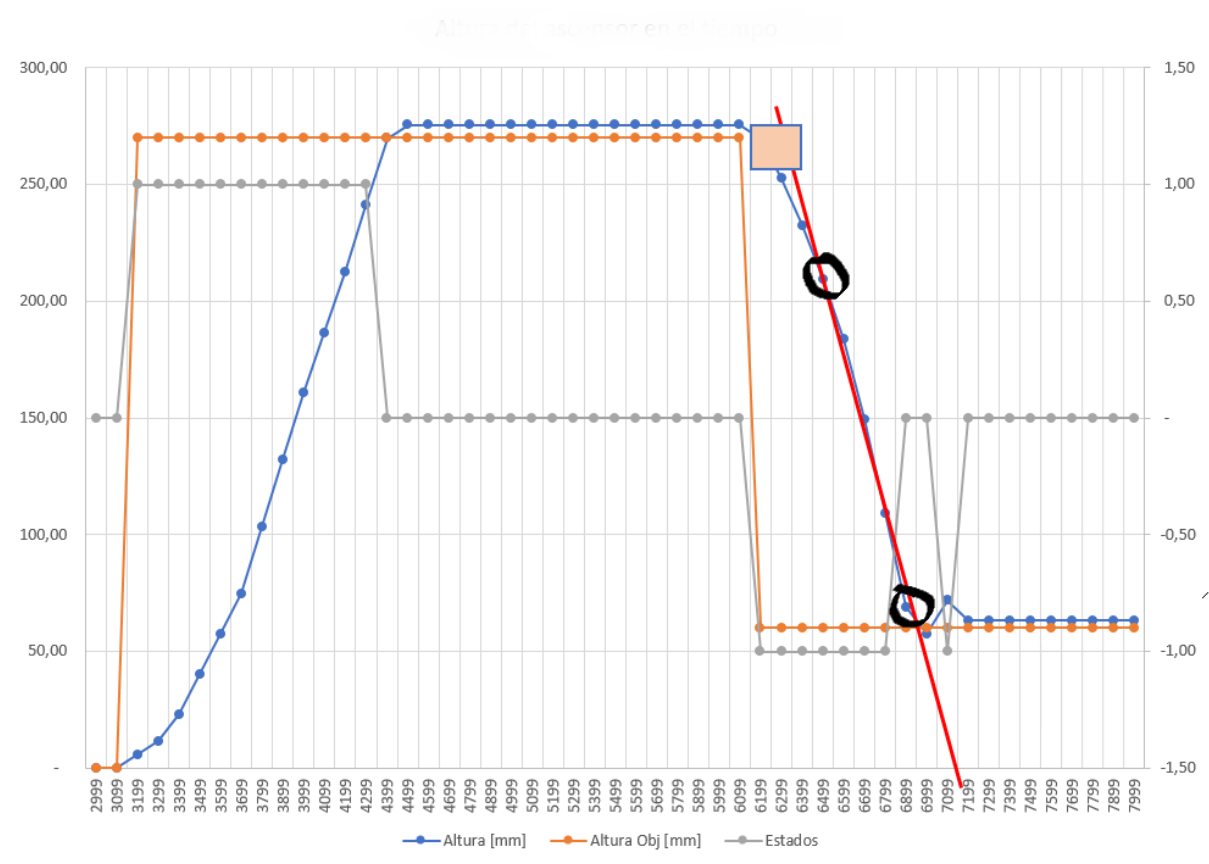


Figura 17: Datos para cálculo de velocidad de bajada.

Para la velocidad de bajada podemos plantear la recta entre estos dos puntos y calcular de la misma manera que para la velocidad de subida.

Ponemos estos datos en otra tabla de Excel y tenemos lo siguiente

“Tabla 4: Datos utilizados para calcular la velocidad de bajada”;

Tiempo [s]	Altura [mm]
6,499	209,51
6,599	183,68
6,699	149,24
6,799	109,06
6,899	68,88

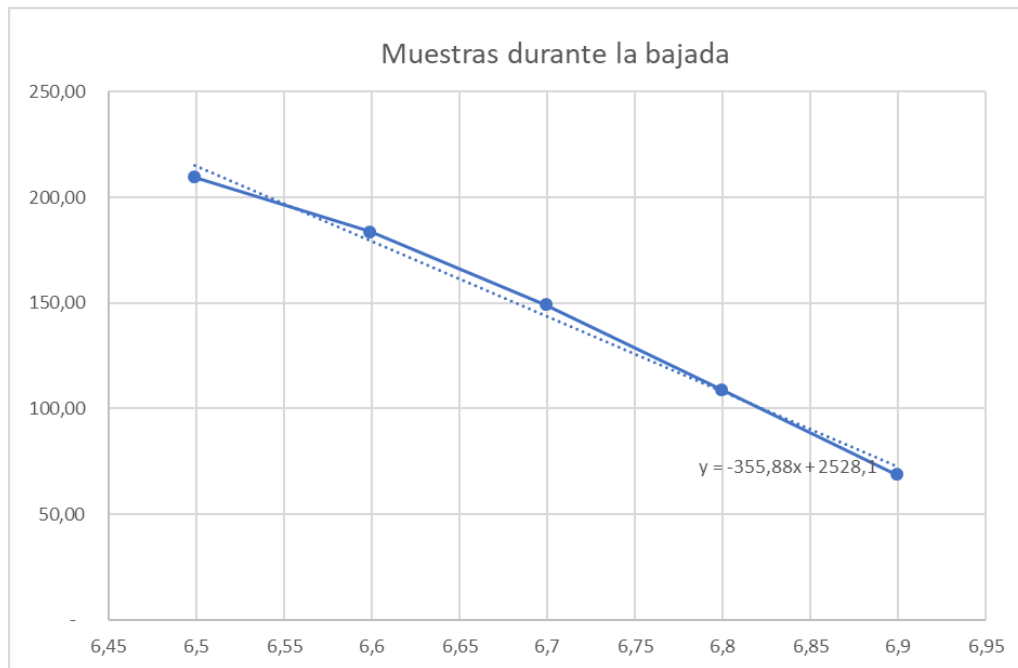


Figura 18: Muestras durante la bajada del ascensor.

Observando el gráfico, la velocidad de bajada es de $35,58 \text{ cm/s}$.

Finalmente para calcular el tau de bajada, lo que tenemos que hacer es analizar cuando la recta de bajada corta la altura $27,552 \text{ cm}$.

Esto es $275,52 \text{ mm} = -355,88 \frac{\text{mm}}{\text{s}} * t_0 + 2528,1 \text{ mm}$

$$t_0 = \frac{275.52 \text{ mm} - 2528.1 \text{ mm}}{-355.88 \text{ mm/s}}$$

$$t_0 = 6320 \text{ ms}$$

Ahora que tenemos este tiempo, para calcular el tau, tenemos que ver la diferencia entre el último momento en que estuvo detenido y este tiempo.

Viendo el gráfico, el último tiempo en que estuvo detenido fue a los 6099 ms

$$\tau = 6320 \text{ ms} - 6099 \text{ ms}$$

$$\tau = 221 \text{ ms}$$

Ya tenemos los parámetros finales de nuestro sistema.

$$v_{subida} = 27,6 \text{ cm/s}$$

$$v_{bajada} = 35,58 \text{ cm/s}$$

$$\tau_{subida} = 326 \text{ ms}$$

$$\tau_{bajada} = 221 \text{ ms}$$

Luego de obtener todos los parámetros del sistema, podemos dejar planteada nuestra función de transferencia. Para esto, utilizamos la velocidad de bajada, y el tau de bajada, utilizamos estos valores ya que son los más grandes.

$$G(s) = \frac{K_v}{\tau * s + 1}$$

$$K_v = \frac{35,58 \text{ cm/s}}{6V}$$

$$\tau = 221 \text{ ms}$$

$$G(s) = \frac{5,93 \frac{\text{cm}}{\text{s}} / V}{221 \text{ ms} * s + 1}$$

Conclusión

Como cierre, podemos decir que logramos concluir el trabajo de laboratorio con éxito haciendo que nuestro control ON-OFF funcione correctamente de acuerdo a la altura medida y el objetivo determinado.

Una de las cosas que nos faltó probar fue el pulsador que por falta de tiempo no pudimos llegar a probar. Este pulsador lo que hace es detener el sistema completamente, siendo la única forma de que funcione nuevamente reiniciarlo.

Si bien ya habíamos analizado que nuestro sistema era no lineal, en los gráficos se puede ver claramente que contamos con un tiempo de crecimiento que es ampliamente inferior al de decrecimiento, es decir, que es muchísimo más rápido. Esto se debe a la diferencia que existe entre el contrapeso y el ascensor.

A pesar de la aparente buena respuesta ante las perturbaciones y del diseño sencillo que permite que nuestro sistema sea estable, nuestra opinión el optoacoplador tiene entre sus dientes $5mm$ de distancia y eso resta precisión que es indeseable para lo que se quiere controlar.

Es importante destacar además que durante la práctica tuvimos sobresaltos con la codificación del programa para correr el arduino, fue sujeto a modificaciones durante el proceso con cosas que no se habían detectado previamente.

A pesar de los desafíos que surgieron durante el desarrollo, en general, la experiencia resultó muy enriquecedora y positiva para todos nosotros. No solo representó el primer proyecto de control automático que emprendimos, sino que también nos obligó a afrontar diversos obstáculos a medida que avanzábamos. Afortunadamente, superamos estos desafíos y conseguimos completar con éxito nuestro trabajo de laboratorio, lo que dejó a todos nosotros muy satisfechos.

Mejoras a desarrollar

Como posibles mejoras a desarrollar en base a la experiencia, podría ser la modificación del código para hacer que el sistema funcione continuamente durante todo el tiempo que se desee y no solamente baje y suba una única vez.

A nivel físico se podría mejorar el peso del ascensor para que sea el mismo cuando suba y cuando baje.

Anexo

A continuación, se coloca el código fuente elaborado para el control automático ON-OFF que se usó para el ascensor.

```

1  // El equipo se inicia.
2  #define I1 8 // Control para el pin 1 del motor A // Baja al alimentar el IN1
3  #define I2 9 // Control para el pin 2 del motor A // Sube el motor al alimentar IN2
4  #define E1 10 // Defino un pin para un botón de emergencia
5  #define O1 12 // Defino un pin para leer el optoacoplador
6
7  // Defino variables globales
8
9  // Se establece una variable ALTURA en 0 (la cual será el destino del ascensor)
10 // Definimos una variable MOTOR que nos indique 0 si esta quieto, 1 si sube y 2 si baja
11 // Se establece una variable ALT_OBJ en 0
12 // El ascensor estará en la planta baja.
13
14 float ALTURA = 0.0; // unificar unidades de altura y tipo de variable
15 enum TIPO_MOTOR {SUBIENDO,DETENIDO,BAJANDO};
16 TIPO_MOTOR MOTOR = DETENIDO; // Estado inicial del motor
17 float ALT_OBJ = 0;
18 bool ESTADO = false; // Leemos si el optoacoplador está tapado o no por una de las astas de la polea (true si esta tapado)
19 float DISTANCIA = 2.87; // distancia que implica un cambio de estado (de 0 a 1 ó de 1 a 0)
20 // Definimos la distancia que recorre el ascensor cada vez que se mueve de una asta hasta la otra de la polea.
21 const float HISTERESIS = 10; // Definimos una histeresis de 10mm
22 const unsigned long DELTA_TIEMPO = 100;
23 static unsigned long lastTime = 0;
24 unsigned long currentTime = 0;
25
26 // Cambios de estados y desplazamiento:
27 // La polea tiene 46 cambios de estados en una vuelta.
28 // Diametro de polea = 42 mm => Recorrido en una vuelta pi*diametro = 131 mm
29 // Cada cambio de estado = 131 mm / 46 = 2,87 mm
30
31 void mostrar();
32 void detenerMaquina();
33 void controlar();
34 void actuar();
35 void lecturaOptoacoplador();
36
37 void setup() {
38     pinMode(O1, INPUT);
39     pinMode(I1, OUTPUT); //Se inician los pines
40     pinMode(I2, OUTPUT);
41     pinMode(E1, OUTPUT);
42     delay (3000); // Luego de 3 segundos arrancamos el programa.
43     ESTADO = digitalRead(O1); // verificar
44     // inicialicen la conexión serial y manden encabezado de los datos
45     Serial.begin(57600);
46     Serial.println("Encabezado de datos");
47
48     lastTime = millis();
49     currentTime = lastTime;
50     mostrar();
51
52
53 }
```

Trabajo de Laboratorio 1: Control Encendido-Apagado de un Ascensor

```
55 void loop() {
56
57     // Verificar el botón de emergencia (E1)
58     if (digitalRead(E1) == HIGH) {
59         detenerMaquina(); // parada de emergencia!!!
60     }
61
62     lecturaOptoacoplador(); // Leer estado optoacoplador
63     controlar(); // Realizar Control
64     actuar(); // Actuar
65     // if pasó 500ms => escribir datos: tiempo / altura objetivo / altura medida / acción de control
66     // Verificar si ha pasado un cierto tiempo (por ejemplo, 500ms)
67
68     currentTime = millis();
69     if (currentTime - lastTime >= DELTA_TIEMPO) {
70         lastTime += DELTA_TIEMPO;
71         mostrar();
72         // Cambio el objetivo luego de 3 segundos
73         // Crear condicional donde si paso mas de 3 segundos cambiar variable ALT_OBJ
74         if (currentTime >= 3000 && ALT_OBJ == 0) {
75             ALT_OBJ = 270; // Cambiar el valor de ALT_OBJ después de 3 segundos
76         }
77         if (currentTime >= 6000 && ALT_OBJ == 270) {
78             ALT_OBJ = 60; // Cambiar el valor de ALT_OBJ después de 3 segundos
79         }
80     }
81 }
82
83 // Parada de emergencia
84 void detenerMaquina(){
85     digitalWrite(I1, LOW);
86     digitalWrite(I2, LOW); // Se establece ambos pines del motor en BAJA para detener el motor.
87     MOTOR = DETENIDO;
88     //Do {
89     | // led titilante
90     //} while (1); // me quedo aca esperando a que alguien haga algo
91 };
```

Trabajo de Laboratorio 1: Control Encendido-Apagado de un Ascensor

```
94 void lecturaOptoacoplador() {
95     static TIPO_MOTOR ultima_accion = DETENIDO;
96     static unsigned long lastChangeTime = 0; // Inicializar el tiempo del último cambio
97
98     // leer pin O1...
99     bool lectura = digitalRead(O1);
100
101     // Verificamos la entrada del optoacoplador e igualamos la variable ESTADO
102     if (ESTADO != lectura) { //hubo un cambio de estado!!!
103
104         // ANTIRREBOTE: Aca tendra que verificar que el el ultimo camio de estado no fue muuuuy reciente...
105         currentTime = millis(); // Obtener el tiempo actual
106         // Verificar si ha pasado un tiempo mínimo (por ejemplo, 10ms) desde el último cambio
107         if (currentTime - lastChangeTime >= 1) {
108             lastChangeTime = currentTime; // Actualizar el tiempo del último cambio
109             ESTADO = lectura; // Actualizar el estado
110
111             // Actualizar la última acción si el motor estaba subiendo o bajando
112             if (MOTOR == SUBIENDO) {
113                 ultima_accion = SUBIENDO;
114                 //ALTURA += DISTANCIA;
115             }
116             else if (MOTOR == BAJANDO) {
117                 ultima_accion = BAJANDO;
118                 //ALTURA -= DISTANCIA;
119             }
120
121             //if (MOTOR==subiendo) {ultima_accion==subiendo;};
122             //if (MOTOR==bajando) {ultima_accion==bajando;};
123
124             if (ultima_accion==SUBIENDO) {ALTURA+=DISTANCIA;};
125             if (ultima_accion==BAJANDO) {ALTURA-=DISTANCIA;};
126
127         }
128     }
129 };
130
131 void controlar(){
132
133     if(ALTURA <= ALT_OBJ + HISTERESIS && ALTURA >= ALT_OBJ - HISTERESIS){
134         MOTOR=DETENIDO;
135     }
136
137     else{
138         if(ALTURA <= ALT_OBJ - HISTERESIS){
139             MOTOR=SUBIENDO;
140         }; // Si estoy más abajo de la altura objetivo menos la hesteresis tengo que subir
141         if(ALTURA >= ALT_OBJ + HISTERESIS){
142             MOTOR=BAJANDO;
143         }; // Si estoy más arriba de la altura objetivo más la hesteresis tengo que bajar
144     };
145 };
146
147 void actuar(){
148     if(MOTOR==SUBIENDO){
149         digitalWrite(I1, LOW);
150         digitalWrite(I2, HIGH); // Se establece el pin 1 del motor en ALTA y el 2 en BAJA
151     }
152     else if(MOTOR==BAJANDO){
153         digitalWrite(I1, HIGH);
154         digitalWrite(I2, LOW); // Se establece el pin 1 del motor en BAJA y el 2 en BAJA
155     }
156     else{
157         digitalWrite(I1, LOW);
158         digitalWrite(I2, LOW); // Se establece ambos pines del motor en BAJA para detener el motor.
159     }
160 }
161
162 }
```

```
164 void mostrar() {
165     // Imprimir los valores de las variables
166     //Serial.print("Tiempo: ");
167     Serial.print(lastTime);
168     Serial.print( "\t");
169     //Serial.print("ALTURA actual: ");
170     Serial.print(ALTURA);
171     Serial.print(" mm \t");
172     Serial.print(ALT_OBJ);
173     Serial.print(" mm \t");
174     //Serial.print(ESTADO);
175     //Serial.print(" mm \t Estado del MOTOR: ");
176
177
178     switch (MOTOR) {
179     case SUBIENDO:
180         Serial.println("SUB");
181         break;
182     case DETENIDO:
183         Serial.println("DET");
184         break;
185     case BAJANDO:
186         Serial.println("BAJ");
187         break;
188     }
189
190     //Serial.println("-----");
191
192 }
```

Bibliografía

Modulo Puente H L298n.

<https://www.eneka.com.uy/robotica/motores/controladores-motor/m%C3%B3dulo-controlador-puente-h-l298n-detail.html>

<https://www.aranacorp.com/es/uso-de-un-modulo-l298n-con-arduino>

Arduino Uno.

https://es.wikipedia.org/wiki/Arduino_Uno

<https://www.xataka.com/basics/que-arduino-como-funciona-que-puedes-hacer-uno>

Pulsador.

<https://www.areatecnologia.com/electricidad/pulsador.html>

Optoacoplador

<https://hetpro-store.com/TUTORIALES/optoacoplador/>

Modelización 1er Orden.

https://ead.undav.edu.ar/pluginfile.php/1992531/mod_resource/content/2/2.a%20-%20Modelizaci%C3%B3n%201er%20orden%20-%202019-08.pdf